

Project Title

Soccer Match & Team Performance Dashboard: An ETL Pipeline Powered by the football-data.org API

1. Introduction / Background

Soccer is the world's most followed sport, generating enormous amounts of structured data with every match including goals, assists, standings shifts, and form trends that millions of fans and analysts track week after week. Yet for most people, accessing that data in a clean, queryable, and visual format remains difficult. League websites bury stats behind multiple clicks, and commercial data platforms are expensive.

The football-data.org API addresses this gap by providing a free, well-documented API covering major competitions including the English Premier League, La Liga, Bundesliga, Serie A, and others. It exposes endpoints for competition standings, match results, team squads, and top scorers structured as clean JSON responses that map naturally to relational tables.

This project is a strong fit because the domain is familiar, the storytelling is intuitive, and the data is rich with KPIs. The analysis produced by this pipeline would be directly useful to:

- Soccer fans wanting a clean, consolidated stats view across a full season
- Fantasy football players tracking goal scorers and team form
- Students and analysts learning hands-on data engineering with a real-world dataset
- Sports writers or bloggers needing data-driven content without a subscription

2. Problem Statement

Soccer statistics are publicly available but not easily accessible in a structured, analysis-ready format. Raw match data is scattered across sources, standings update constantly throughout a season, and there is no free interactive dashboard that consolidates team performance trends into one place such as win/loss patterns, home vs. away splits, goals scored vs. conceded, and scorer leaderboards.

This project solves that problem by building a fully automated ETL pipeline that extracts live competition data from football-data.org, transforms it into clean relational tables, stores it in a local database, and presents it through an interactive Dash dashboard which gives any user immediate access to meaningful soccer insights with no manual effort.

3. Objectives

- Extract competition standings, match results, and top scorer data from the football-data.org v4 REST API using Python and the requests library
- Parse and flatten nested JSON responses into structured tabular DataFrames using pandas
- Clean and validate the data: handle null scores (postponed/cancelled matches), standardize column names and date formats, remove duplicates, and compute derived metrics
- Calculate key performance indicators (KPIs): goal difference, points per game, home vs. away win rates, and rolling 5-match form

- Load processed data into a SQLite database using SQLAlchemy, with separate tables for standings, matches, scorers, and teams
- Build an interactive Dash (Plotly) dashboard with at least four visualizations and dropdown filtering by competition and team
- Document all scripts clearly with comments so the pipeline is understandable and reproducible

4. Methodology / Technical Approach

API & Data Source

- API: football-data.org - <https://www.football-data.org/>
- Version: v4 (released May 2022, current stable version)
- Auth: Free registration required; API token passed as X-Auth-Token header
- Rate Limit: Free tier: 10 requests/minute. Monitored via X-Requests-Available-Minute response header

Key endpoints used:

- GET /v4/competitions/{id}/standings - current league table (total, home, away)
- GET /v4/competitions/{id}/matches - all match results for a season, filterable by matchday or date range
- GET /v4/competitions/{id}/scorers - top goal scorers, descending by goals
- GET /v4/teams/{id}/matches - team-specific match history for home/away analysis

Target competitions (free tier): Premier League (PL), La Liga (PD), Bundesliga (BL1), Serie A (SA)

Tools & Technologies

- Python 3.x - primary programming language
- requests - HTTP GET calls to the football-data.org API; handles headers and response parsing
- pandas - JSON flattening, DataFrame manipulation, KPI calculation, data cleaning
- SQLite + SQLAlchemy - lightweight local relational database; no server setup required
- Dash (Plotly) - Python-native interactive web dashboard framework
- python-dotenv - secure storage of API token in a .env file (not hardcoded)
- VS Code - development environment

ETL Workflow Design

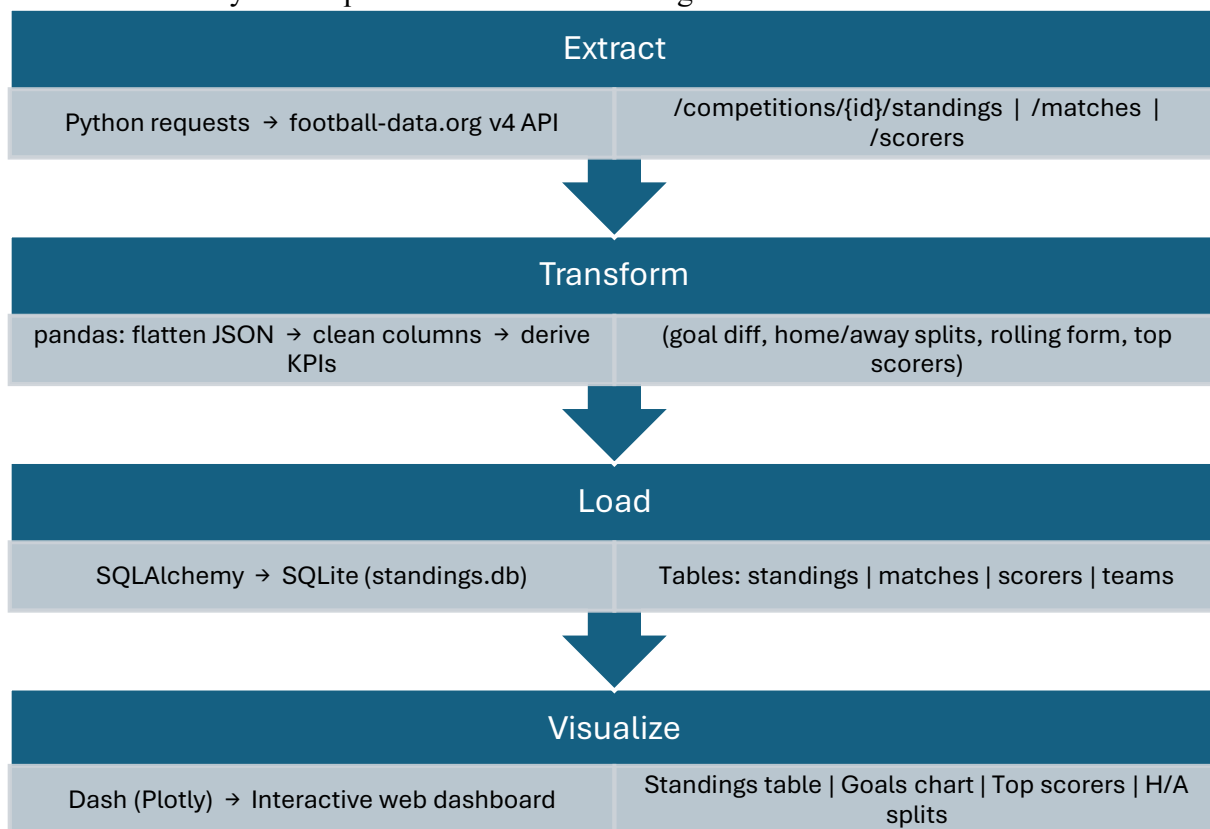
The pipeline is organized into three modular scripts plus a dashboard app:

- `extract.py` - calls each API endpoint, respects rate limits with `time.sleep()`, saves raw JSON to `/data/raw/` as backup
- `transform.py` - reads raw JSON, flattens nested objects (team, area, score), renames columns, parses dates, fills nulls, computes goal difference and form strings
- `load.py` - connects to SQLite via SQLAlchemy, creates/replaces four tables: standings, matches, scorers, teams
- `app.py` - Dash application that reads from SQLite and renders charts with dropdown interactivity

ETL Architecture Diagram

The following diagram shows the end-to-end data flow from API to dashboard:

- Extract: Python requests → football-data.org v4 API Load



Visualization Strategy

The Dash dashboard will include four core views:

- League Standings Table - interactive sortable table showing rank, team, played, wins, draws, losses, GF, GA, GD, points; filterable by competition
- Goals Scored vs. Conceded - grouped bar chart comparing attacking and defensive performance per team across the season
- Top Scorers Leaderboard - horizontal bar chart showing leading goal scorers, colored by team; filterable by competition
- Home vs. Away Performance - side-by-side bar chart showing win rate, average goals, and points per game split by home/away for any selected team

All charts will share a Competition dropdown (Premier League, La Liga, etc.) and a Season selector. The team-level view will include an additional Team dropdown. Layouts will be clean, mobile-friendly, and styled consistently with Plotly's built-in themes.

5. Timeline

Week	Tasks & Deliverables
Week 1	Register for football-data.org free API key; explore v4 endpoint documentation; define target competitions (e.g. Premier League, La Liga); finalize proposal; set up Python environment with requests and pandas
Week 2	Write extract.py to call /competitions/{id}/standings, /competitions/{id}/matches, and /competitions/{id}/scorers endpoints; save raw JSON to backup files; handle rate-limit headers (10 req/min free tier)
Week 3	Write transform.py using pandas: flatten nested JSON, normalize column names, parse dates, calculate derived KPIs (goal difference, home vs away splits, form strings); write load.py to store to SQLite via SQLAlchemy
Week 4	Build Dash app (app.py): league standings table, goals scored vs conceded bar chart, top scorers chart, home/away performance comparison; add season and team dropdown filters
Week 5	End-to-end testing; fix edge cases (null scores, postponed matches); polish dashboard layout; write README documentation; prepare final presentation and project submission

6. Expected Outcomes

Upon completion, the project will deliver the following:

- A working, modular Python ETL pipeline (extract.py, transform.py, load.py) that pulls soccer data from football-data.org and loads it into a local SQLite database
- A clean SQLite database containing four structured tables: standings, matches, scorers, and teams, populated with at least one full competition season of data
- An interactive Dash web dashboard with four visualizations (standings table, goals chart, top scorers, home/away splits) and competition/team dropdown filters
- A well-documented GitHub-style project folder with a README explaining setup, API key configuration, and how to run the pipeline
- A final presentation and live demo of the working dashboard, walking through the pipeline design and key insights from the data

This project is intentionally scoped to be fully completable within five weeks. The football-data.org free tier provides sufficient data for all planned visualizations, and each pipeline stage is well-isolated, making it easy to build and test incrementally. The end result will be a polished, technically complete data engineering project that demonstrates every layer of a modern ETL workflow.